

Member 1 Report: Business Problem and Data Preparation

1. Introduction

Food delivery platforms must make fast operational decisions under uncertain urban conditions. During peak hours, many orders arrive within a short time window, while riders, restaurant locations, customer locations, traffic, and weather conditions all affect delivery efficiency. Poor assignment and routing decisions can increase travel distance, delivery time, rider workload imbalance, and late deliveries.

This project studies a prescriptive analytics problem for food delivery operations: given a set of orders in the same city and operating period, how should a platform assign orders to available riders and determine delivery routes to reduce total delivery distance and estimated delivery time?

The project uses the Zomato Delivery Operations Analytics Dataset. The dataset contains order-level information, including restaurant coordinates, delivery coordinates, order time, pickup time, traffic density, weather conditions, rider information, vehicle type, and actual delivery time. These fields allow the group to build a simplified vehicle routing and order assignment problem.

2. Business Problem

The main business problem is:

In a selected city and operating period, how should a food delivery platform assign orders to a limited number of riders and optimize each rider's delivery sequence to minimize total travel distance, estimated delivery time, and potential service delay?

The decision problem has two parts:

1. **Order assignment:** decide which rider should serve each order.
2. **Route sequencing:** decide the order in which each rider should deliver assigned orders.

The primary objective is to minimize total delivery distance. A secondary objective is to reduce estimated delivery time by considering traffic and weather conditions. The optimized solution will be compared with simple non-optimized baselines to evaluate operational improvement.

3. Data Description

The raw Zomato dataset contains approximately 45,584 order records and 20 fields. The most relevant fields are:

Category	Fields	Use in Project
Order identifier	ID	Identify each order
Rider information	Delivery_person_ID	Extract real city code and rider information
Restaurant location	Restaurant_latitude , Restaurant_longitude	Route start or pickup location
Customer location	Delivery_location_latitude , Delivery_location_longitude	Delivery destination
Time information	Order_Date , Time_Orderd , Time_Order_picked	Select date and peak-hour period
Operating conditions	Weather_conditions , Road_traffic_density	Adjust estimated travel time
Vehicle information	Type_of_vehicle , Vehicle_condition	Support speed or capacity assumptions
Delivery outcome	Time_taken (min)	Reference for interpreting delivery time

One important data issue is that the `City` field does not represent a real city name. It only describes city type, such as `Metropolitan` , `Urban` , or `Semi-Urban` . Therefore, this project extracts the real city code from `Delivery_person_ID` . For example, `JAPRES14DEL02` is treated as a Jaipur order, with city code `JAP` .

This step is essential because routing optimization must be performed within one real geographic area. Orders from different cities cannot be mixed in one route optimization problem.

4. Data Cleaning

The following cleaning rules are applied before creating the optimization datasets:

1. Remove invalid coordinates

Records are removed if any of the restaurant or delivery latitude/longitude values equals 0. These records cannot support meaningful distance calculation.

2. Remove missing key fields

Records with missing `Time_Orderd` , `Weather_conditions` , `Road_traffic_density` , or `City` are removed because these fields are required for time-period filtering and operational interpretation.

3. Extract real city code

A new field, `city_code`, is extracted from `Delivery_person_ID`. This is used to select one actual city for each optimization scenario.

4. Convert date and time fields

`Order_Date`, `Time_Orderd`, and `Time_Order_picked` are converted into standard date/time fields. An additional `order_hour` field is created to filter peak-hour orders.

5. Select city-period samples

Each experimental dataset must come from one city, one date, and one operating period. The main period used in this project is the evening peak from 17:00 to 23:00.

After cleaning, the dataset remains large enough for scenario selection and robustness testing.

5. Experimental Dataset Design

Member 1 provides three types of datasets. Each dataset has a clear purpose and should not be mixed with the others.

Dataset A: Main Optimization Dataset

The main dataset is used for the core optimization model.

Item	Setting
Main city	JAP / Jaipur
Time window	One selected date, evening peak 17:00-23:00
Order sample size	50 orders if available
Suggested riders	5 riders
Suggested rider capacity	8-10 orders per rider

This dataset answers the main question:

In the Jaipur evening peak scenario, how can 50 orders be assigned to 5 riders and sequenced to reduce total delivery distance or estimated delivery time?

Member 2 should use this dataset to formulate the main mathematical model. Member 3 should use it for the main computational experiment, route visualization, and baseline comparison.

Dataset B: Baseline Dataset

The baseline is derived from the same Jaipur main dataset. It is not a separate city or separate business problem.

The recommended baselines are:

- 1. **Original order baseline:** assign and deliver orders according to their original order sequence.
- 2. **Random assignment baseline:** randomly assign orders to riders and use random delivery order.
- 3. **Simple geographic baseline:** group orders by geographic proximity but do not optimize delivery sequence.

This dataset answers:

Does the optimization model improve delivery performance compared with simple non-optimized operating rules?

Recommended comparison metrics include total distance, estimated delivery time, average distance per rider, workload imbalance, and late orders.

Dataset C: Robustness / Transferability Datasets

Robustness datasets are used to test whether the same optimization approach works in other cities. They should be solved as independent scenarios, not combined with Jaipur.

The recommended robustness cities are:

City code	City
MUM	Mumbai
HYD	Hyderabad

Each robustness dataset follows the same design:

Item	Setting
Time window	Evening peak 17:00-23:00
Order sample size	50 orders if available
Suggested riders	5 riders

These datasets answer:

Is the proposed assignment and routing method still effective under different city spatial structures?

The final report can compare the improvement rate across Jaipur, Mumbai, and Hyderabad.

6. Assumptions

The following assumptions define the scope of the project:

- 1. All orders in one optimization scenario belong to the same real city.
- 2. Each order must be served exactly once by one rider.
- 3. Each rider has a maximum order capacity.
- 4. Restaurant and customer coordinates are treated as accurate after cleaning.
- 5. Haversine distance is used to approximate travel distance.
- 6. Traffic and weather conditions can be represented by travel-time multipliers.
- 7. The project does not model full real-world VRPTW complexity.
- 8. The main model is a simplified assignment and routing problem.
- 9. Robustness datasets are used only for transferability checks and are not mixed with the main city.

7. Deliverables to Other Members

Member 1 provides the following files:

File	Purpose
data/processed/cleaned_zomato_full.csv	Cleaned full dataset with extracted city code
data/processed/main_optimization_jaipur.csv	Main optimization dataset
data/processed/robustness_mumbai.csv	Robustness dataset for Mumbai
data/processed/robustness_hyderabad.csv	Robustness dataset for Hyderabad
data/processed/member1_data_summary.csv	Summary table for selected datasets

Member 2 should build the mathematical model mainly around `main_optimization_jaipur.csv` . Member 3 should generate baselines, run optimization, visualize routes, and repeat the same workflow on the robustness datasets.